PL/SQL Exercise 3

Scenario 1: The bank needs to process monthly interest for all savings accounts. Question: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
create or replace procedure ProcessMonthlyInterest
is
begin
   update Accounts set Balance = Balance + (Balance * 1 / 100), LastModified = sysdate
   where AccountType = 'Savings';

   commit;
   dbms_output.put_line('Monthly interest processed successfully.');
```

```
exception
   when others then
      rollback;
      dbms_output.put_line('Error : ' || sqlerrm);
end;
/
```

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance. Question: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
create or replace procedure UpdateEmployeeBonus(
   dept varchar2,
   bonusPer number
)
is
begin
   update Employees set Salary = Salary + (Salary * bonusPer / 100)
   where Department = dept;

   if sql%rowcount = 0 then
      rollback;
      dbms_output.put_line('Department Not Found');
   else
      commit;
      dbms_output.put_line('Employee Bonus Updated Successfully');
   end if;
```

```
exception
   when others then
      rollback;
      dbms_output.put_line('Error : ' || sqlerrm);
end;
/
```

Scenario 3: Customers should be able to transfer funds between their accounts. Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
create or replace procedure TransferFunds(
   source_accId number,
   dest_accId number,
   transferAmt number
)
is
   bal number;
begin
   select Balance into bal from Accounts where AccountID = source_accId;
   if bal >= transferAmt then

      update Accounts set Balance = Balance - transferAmt, LastModified = sysdate
      where AccountID = source_accId;

      update Accounts set Balance = Balance + transferAmt, LastModified = sysdate
      where AccountID = dest_accId;

      commit;
      dbms_output.put_line('Funds Transferred Successfully');

   else
      raise_application_error(-20001,'Insufficient Funds');
   end if;

exception
   when no_data_found then
      rollback;
      dbms_output.put_line('Account Not Found');

   when others then
      rollback;
      dbms_output.put_line('Error : ' || sqlerrm);
end;
/
```